Objetivos:

- Desenvolver um aplicativo Flutter para coleta e visualização de localizações.
- Gerenciar permissões e coletar posições continuamente por meio de streams.
- Armazenar, consultar e excluir registros em SQLite.
- Apresentar as localizações em uma lista e em um mapa interativo.
- Organizar o projeto com separação de responsabilidades.
- Criar e configurar um ícone próprio para o aplicativo.

Observação:

- **Valor:** 0,5 ponto na média final. A partir da sua entrega haverá uma prova individual, no papel e sem consulta com peso de 1,5 ponto na média final.
- **Entrega:** pode ser individual ou em dupla.
- **Data da entrega:** 04/11/2026. A data da avaliação será dia 10/11/26.
- **Forma de entrega:** enviar o link do repositório para o professor pela tarefa no Teams até 02/11/2026. No repositório deverá constar os nomes dos integrantes.

Objetivo

Desenvolver um aplicativo Flutter para Android/iOS que permita iniciar e interromper a coleta da localização do aparelho, armazenar as posições em um banco SQLite e consultar os registros em uma lista e em um mapa interativo.

A atividade deverá demonstrar o uso de permissões, programação assíncrona, streams (usados na coleta contínua de localização), persistência local, navegação entre telas e separação de responsabilidades.

Estrutura sugerido do projeto

Organize o código por funcionalidade e, dentro dela, por responsabilidade. Utilize a seguinte estrutura mínima:

```
assets/
└── icons/
    └── localizacoes.png            # Imagem original do ícone
android/app/src/main/
├── AndroidManifest.xml             # Referências ao ícone do aplicativo
└── res/
    ├── drawable-nodpi/
    │   └── icone_localizacoes.png    # Imagem usada pelo ícone adaptativo
    ├── mipmap-mdpi/ic_localizacoes.png
    ├── mipmap-hdpi/ic_localizacoes.png
    ├── mipmap-xhdpi/ic_localizacoes.png
    ├── mipmap-xxhdpi/ic_localizacoes.png
    ├── mipmap-xxxhdpi/ic_localizacoes.png
```

```
    ├── mipmap-anydpi-v26/
    │   └── ic_localizacoes.xml       # Definição do ícone adaptativo
    └── values/
        └── icone_localizacoes.xml    # Cor de fundo do ícone
lib/
├── main.dart
├── app.dart
└── features/
    └── localizacoes/
        ├── models/
        │   └── localizacao.dart
        ├── data/
        │   ├── database/
        │   │   └── banco_localizacoes.dart
        │   └── repositories/
        │       └── repositorio_localizacoes.dart
        ├── services/
        │   ├── servico_localizacao.dart
        │   └── cache_mapa.dart
        └── presentation/
            ├── screens/
            │   ├── tela_coleta.dart
            │   ├── tela_localizacoes.dart
            │   └── tela_mapa.dart
            └── widgets/
                ├── item_localizacao.dart
                └── mapa_localizacoes.dart
```

Responsabilidades dos arquivos

Arquivo	Responsabilidade
main.dart	Inicializar o Flutter e executar o widget raiz.
app.dart	Configurar o MaterialApp e definir a tela inicial.
localizacao.dart	Representar um registro tipado com identificador, latitude, longitude, precisão e instante; converter os dados para e de mapas usados na persistência.
banco_localizacoes.dart	Abrir o banco, definir sua versão e criar a tabela. Reutilizar a conexão e permitir nova tentativa se a abertura falhar.
repositorio_localizacoes.dart	Centralizar as operações de inserir, consultar e excluir todos os registros. Retornar objetos Localizacao nas consultas.
servico_localizacao.dart	Verificar o GPS e as permissões, fornecer o stream de posições e abrir as configurações de permissão do aplicativo.

Arquivo	Responsabilidade
cache_mapa.dart	Preparar o armazenamento local das imagens do mapa antes de carregá-las, utilizando uma pasta privada do aplicativo.
tela_coleta.dart	Apresentar os controles, gerenciar a assinatura do stream e coordenar a gravação por meio do repositório.
tela_localizacoes.dart	Consultar e apresentar o histórico, permitir sua atualização e confirmar a exclusão dos dados.
tela_mapa.dart	Carregar os registros e tratar os estados de espera, erro e histórico vazio antes de apresentar o mapa.
item_localizacao.dart	Apresentar os dados de uma localização, podendo ser reutilizado na lista e nos detalhes do mapa.
mapa_localizacoes.dart	Exibir a camada de ruas, os marcadores e as interações do mapa.

Arquivos adicionais são permitidos quando houver uma responsabilidade clara. Não concentre toda a implementação no `main.dart` e não crie camadas sem uso apenas para preencher a estrutura.

Requisitos do aplicativo

1. Tela inicial — coleta de localização

O aplicativo deverá apresentar uma tela inicial com:

- Botões **Iniciar coleta**, **Parar coleta**, **Consultar histórico**, **Ver mapa** e **Configurações de permissão**.

Ao iniciar a coleta, o aplicativo deverá:

- Verificar se o serviço de localização está habilitado.
- Verificar e solicitar a permissão de localização, quando necessário.
- Orientar o usuário caso a permissão seja negada ou precise ser habilitada nas configurações.
- Utilizar um **stream de posições**, com alta precisão e intervalo solicitado de **20 segundos**.
- Salvar cada posição recebida no SQLite.
- Evitar assinaturas duplicadas e gravações sobrepostas.

No Android, a coleta deverá utilizar um serviço em primeiro plano, mantendo uma notificação visível enquanto estiver ativa. O intervalo solicitado poderá variar conforme o sistema operacional e a disponibilidade de localização.

A coleta deverá continuar ao navegar para outras telas e ao minimizar o aplicativo. Ao selecionar **Parar coleta**, a assinatura do stream deverá ser cancelada. Não é obrigatório reiniciar a coleta automaticamente após o encerramento do processo ou a reinicialização do aparelho.

Os botões de iniciar e parar deverão ser habilitados conforme o estado da coleta, evitando operações conflitantes.

2. Tela de histórico

A tela deverá:

- Apresentar as **últimas 100 localizações**, da mais recente para a mais antiga.
- Exibir data e hora locais, latitude, longitude e precisão estimada em metros.
- Formatar latitude e longitude com seis casas decimais e precisão com uma casa decimal.
- Permitir atualizar os registros sem interromper a coleta.

Deverá existir um botão para **limpar todo o histórico**, incluindo os registros anteriores aos 100 exibidos.

Antes da exclusão, apresentar uma confirmação com as opções **Cancelar** e **Limpar histórico**, informando que a operação é permanente. Se a coleta estiver ativa, informar que novos registros continuarão sendo salvos.

Após a exclusão, atualizar a lista e apresentar uma mensagem de sucesso. Em caso de falha, apresentar uma mensagem de erro.

3. Tela de mapa interativo

A tela deverá:

- Apresentar as últimas 100 posições salvas.
- Exibir uma **camada de fundo com ruas e referências geográficas**.
- Representar as localizações por marcadores.
- Destacar a localização mais recente em vermelho.
- Enquadrar os pontos ao carregar os dados.
- Disponibilizar um botão para enquadrar todos os pontos novamente.
- Disponibilizar um botão para atualizar os registros.

4. Persistência local

Utilizar o banco **localizacoes.db**, com a tabela **localizacoes**:

Coluna	Tipo	Requisito
id	INTEGER	Chave primária com autoincremento.
latitude	REAL	Obrigatória.
longitude	REAL	Obrigatória.
precisao	REAL	Obrigatória; valor em metros.
instante	TEXT	Obrigatória; data e hora em UTC, no formato ISO 8601.

Os dados deverão permanecer disponíveis após fechar e reabrir o aplicativo. A apresentação deverá converter os instantes para o horário local.

O limite de 100 registros deverá ser aplicado apenas à consulta, sem excluir automaticamente os registros antigos.

A limpeza deverá remover todas as linhas da tabela, preservando sua estrutura e o arquivo do banco.

5. Ícone do aplicativo

- Criar um ícone próprio, relacionado à proposta do aplicativo.
- Substituir o ícone padrão do Flutter.
- Manter a imagem original em assets/icons/.
- Verificar se o ícone aparece corretamente na tela inicial e na lista de aplicativos após a instalação.

6. Organização e restrições

- Separar apresentação, serviços, modelo e persistência conforme as responsabilidades descritas.
- Não executar SQL diretamente nas telas ou widgets.
- Utilizar objetos Localizacao na apresentação, evitando mapas brutos retornados pelo SQLite.
- Não utilizar Timer.periodic ou workmanager para substituir o stream de localização.
- Não utilizar bancos remotos ou serviços de backend para armazenar as posições.
- Tratar erros de permissão, localização e acesso ao banco.
- Liberar assinaturas e controladores ao encerrar os componentes responsáveis.
- Verificar mounted antes de atualizar a interface após operações assíncronas.
- Manter apenas dependências utilizadas e entregar o código formatado, sem problemas apontados por flutter analyze.
- Respeitar a atribuição e as regras de uso do provedor do mapa, incluindo o armazenamento em cache.
- Não incluir bancos com coordenadas pessoais no repositório entregue.

Caso seja escolhida a plataforma **iOS**, adaptar as permissões, o comportamento em segundo plano e os recursos do ícone às exigências dessa plataforma, mantendo as funcionalidades propostas.